

TROUBLESHOOTING PLAYBOOKS AND MANAGED HOSTS

You can troubleshoot playbook errors depending on the nature of errors.

1:LOG FILES FOR ANSIBLE

- Logs play an important role for troubleshooting.
- But, by default, Red Hat Ansible Engine is not configured to log its output to any log file.
- It provides a built-in logging infrastructure that can be configured through the `log_path` parameter in the default section of the `ansible.cfg` configuration file, or through the `$ANSIBLE_LOG_PATH` environment variable.
- If any or both are configured, Ansible stores output from both the `ansible` and `ansible-playbook` commands in the log file configured, either through the `ansible.cfg` configuration file or the `$ANSIBLE_LOG_PATH` environment variable.
- If you configure Ansible to write log files to `/var/log`, then Red Hat recommends that you configure `logrotate` to manage the Ansible log files.

1. Create the logrotate config file**

Create:

```
vi /etc/logrotate.d/ansible
```

```
/var/log/ansible.log {  
    daily  
    rotate 14  
    compress  
    missingok  
    notifempty  
    create 0640 root root  
    delaycompress  
    sharedscripts  
    postrotate  
        # Reload or restart any service if needed  
        # (Ansible does not need a reload)  
        :  
    endscript  
}
```

2. Explanation of each directive

Directive	Meaning
daily	Rotate the log every day
rotate 14	Keep 14 old logs (14 days)
compress	Compress old logs (.gz)
delaycompress	Compress starting from the <i>second</i> rotation (avoids compressing the most recent rotated file)
missingok	Don't complain if the log file doesn't exist
notifempty	Don't rotate if the file is empty
create 0640 root root	Create a new log file with these permissions
sharedscripts	Ensures postrotate runs only once
postrotate ... endscript	Commands to run after rotation (Ansible doesn't need any)

3. Ensure Ansible is writing logs to /var/log

In `ansible.cfg`:

...

[defaults]

log_path = /var/log/ansible.log

...

Then:

...

sudo touch /var/log/ansible.log

sudo chown root:root /var/log/ansible.log

sudo chmod 0640 /var/log/ansible.log

...

4. Test your logrotate config

...

```
sudo logrotate-d /etc/logrotate.d/ansible
```

...

This performs a dry run.

To force rotation:

...

```
sudo logrotate-f /etc/logrotate.d/ansible
```

...

5. Recommended production settings

If Ansible logs are heavy (automation pipelines, CI/CD, AWX, etc.):

...

```
size 50M
```

```
rotate 30
```

```
daily
```

```
compress
```

...

This rotates when the file hits 50 MB, but still checks daily.

2: THE DEBUG MODULE

- The **debug** module provides insight into what is happening in the play.
- This module can display the value for a certain variable at a certain point in the play.
- This feature is key to debugging tasks that use variables to communicate with each other.
- **The debug module is used to:**
 - Print variables.
 - Print custom messages.
 - Show host facts.

- Display task results.
- Help troubleshoot playbooks.

1:example

```

```yaml
- name: Show OS family for each server

hosts: all

gather_facts: yes

tasks:

 - name: Show custom message with variable

 debug:

 msg: "The server {{ inventory_hostname }} belongs to {{ ansible_os_family }} family"

```

### 2: Example

```

- name: Debug Module

hosts: rhelnode

- name: Show multiple variables

debug:

 msg:

 - "Hostname: {{ inventory_hostname }}"

 - "OS: {{ ansible_distribution }}"

 - "Version: {{ ansible_distribution_version }}"

```

### 3:Example-Display Tasks Results

```

- name: Display task results using debug

hosts: all

gather_facts: no

tasks:

 - name: Run a command

 command: uname-a

 register: cmd_output

 - name: Show full result

 debug:

 var: cmd_output

```

### 3: MANAGING ERRORS:

- There are several issues than can occur during a playbook run, mainly related to the syntax of either the playbook or any of the templates it uses, or due to connectivity issues with the managed hosts (for example, an error in the host name of the managed host in the inventory file).
- Those errors are issued by the ansible-playbook command at execution time.

- **1: use --syntax-check**

```
ansible-playbook site.yml--syntax-check
```

✓ What it catches

- Bad indentation
- Missing colons
- Invalid YAML
- Invalid module arguments
- Missing roles or tasks

- **2: use --step:**

- You can also use the `--step` option to step through a playbook one task at a time.
- The `ansible-playbook--step` command interactively prompts for confirmation that you want each task to run.

```
ansible-playbook site.yml--step
```

- Useful for
- Debugging
- Learning how a playbook flows
- Testing tasks one by one

### 3: use --start-at-task:

- The `--start-at-task` option allows you to start execution of a playbook from a specific task. It takes as an argument the name of the task at which to start.

## PlayBook: site.yml

```
``yaml
```

```
- name: Install and configure NGINX
```

```
 hosts: webservers
```

```
 become: yes
```

```
 gather_facts: yes
```

```
 tasks:
```

```
 - name: Install NGINX
```

```
 package:
```

```
 name: nginx
```

```
 state: present
```

```
 - name: Start and enable NGINX service
```

```
 service:
```

```
 name: nginx
```

```
 state: started
```

```
 enabled: yes
```

```
 - name: Deploy custom index.html
```

```
 copy:
```

```
 dest: /usr/share/nginx/html/index.html
```

```
 content: |
```

```
 <h1>Welcome to NGINX managed by Ansible</h1>
```

```
 - name: Verify NGINX is running
```

```
 command: systemctl status nginx
```

```
 register: nginx_status
```

```
 - name: Show NGINX status output
```

```
 debug:
```

```
 var: nginx_status.stdout
```

```
...
```

Then Run:

```
ansible-playbook site.yml--start-at-task="Install NGINX"
```

You can increase verbosity of `ansible-playbook` command by adding one or more `-v` switch:

### Verbosity Options

Option	Description
<code>-v</code>	Shows basic output. Displays task results in more detail than default.
<code>-vv</code>	Shows both <b>input and output</b> data. Useful for seeing module arguments.
<code>-vvv</code>	Adds <b>connection debugging</b> . Shows SSH details, control socket info, and connection negotiation.
<code>-vvvv</code>	Adds <b>deep debugging</b> . Shows scripts executed on remote hosts, environment variables, and the user executing each script. Also shows internal Ansible operations.

### 4: use `--check` or `-C`:

- You can use the `ansible-playbook--check` or `-C` command to run smoke tests on a playbook.
- This option executes the playbook without making changes to the managed hosts' configuration.
- If a module used within the playbook supports check mode, then the changes that would have been made to the managed hosts are displayed but not performed.
- If `check mode` is not supported by a module, then the changes are not displayed but the module still takes no action.

```
ansible-playbook--check playbook.yml
```

- The `ansible-playbook--check` command might not work properly if your tasks use conditionals.
- You can also control whether individual tasks run in check mode with the `check_mode` setting.
- If a task has `check_mode: yes` set, it always runs in check mode, whether or not you passed the `--check` option to `ansible-playbook`.
- Likewise, if a task has `check_mode: no` set, it always runs normally, even if you pass `--check` to `ansible-playbook`.

## 1: Example

```
``yaml
- name: Demonstrate check_mode: yes

 hosts: all

 gather_facts: no

 tasks:

 - name: This task always runs in check mode

 debug:

 msg: "This task is forced into check mode"

 check_mode: yes

 - name: Show that this task runs normally

 debug:

 msg: "Normal task running"
...

```

## 2: Example

```
...

- name: Demonstrate check_mode: no

 hosts: all

 gather_facts: no

 tasks:

 - name: This task always runs even in check mode

 shell: uname-a

 check_mode: no

 register: uname_output

 - name: Show output

 debug:

 msg: "{{ uname_output.stdout }}"
...

```

## WARNING

Tasks that have `check_mode: no` set will run even when the playbook is run with `ansible-playbook--check`. Therefore, you cannot trust that the `--check` option will make no changes to managed hosts, without confirming this to be the case by inspecting the playbook and any roles or tasks associated with it.

### Why this warning matters

When you run:

...

`ansible-playbook--check`

...

*you expect **\*\*zero changes\*\*** to be made on managed hosts.*

But that expectation is **\*\*NOT guaranteed\*\*** if your playbook contains:

- a. tasks with `check_mode: no`
- b. older playbooks using `always_run: yes` (deprecated)
- c. modules that **don't support check mode** (e.g., `command`, `shell`, `raw`)

This is why Red Hat warns you:

👉 You must inspect your playbooks and roles before trusting `--check`.

#### 1. Example: Task that WILL run even in check mode (`check_mode: no`)

```yaml

- name: Demo check_mode no

hosts: all

gather_facts: no

tasks:

- name: This task WILL run even with--check

shell: echo "Real change happening" >> /tmp/real_change.txt

check_mode: no

...

Run:

...

`ansible-playbook demo.yml--check`

...

✓ This task ****executes for real****

✓ File ``/tmp/real_change.txt`` is modified

✗ So ``--check`` is NOT safe here

2. Example: Task that ALWAYS runs in check mode (``check_mode: yes``)******

```
``yaml
```

```
---
```

```
- name: Demo check_mode yes
```

```
  hosts: all
```

```
  gather_facts: no
```

```
  tasks:
```

```
    - name: This task ALWAYS runs in check mode
```

```
      debug:
```

```
        msg: "This task is forced into check mode"
```

```
      check_mode: yes
```

```
...
```

Run:

```
...
```

```
ansible-playbook demo.yml
```

```
...
```

✓ This task behaves ****as if**** you used ``--check``

✓ It will NOT make changes

✓ Even though you ran normally

5: use --diff:

- The ansible-playbook command also provides a --diff option.
- This option reports the changes made to the template files on managed hosts.
- If used with the --check option, those changes are displayed in the command's output but not actually made.
- --diff shows before and after differences for any file that Ansible modifies. like
 - Lines added
 - Lines removed
 - Lines changed
 - Template differences
 - Config file update

```
ansible-playbook demo.yml--check --diff
```

1:Example

```
'''
```

```
- name: Create initial file
```

```
hosts: all
```

```
gather_facts: no
```

```
tasks:
```

```
- name: Create /tmp/demo.txt with initial content
```

```
copy:
```

```
dest: /tmp/demo.txt
```

```
content: |
```

```
    This is the original file content.
```

```
    Managed by Ansible.
```

```
'''
```

After: Modify the file (for diff demonstration)

```
...  
  
- name: Modify file to demonstrate diff  
  
  hosts: all  
  
  gather_facts: no  
  
  
  tasks:  
  
    - name: Update content inside /tmp/demo.txt  
  
      lineinfile:  
  
        path: /tmp/demo.txt  
  
        line: "This line was added by Ansible during modification."  
...
```

Now Run:

```
ansible-playbook modify_file.yml--check --diff
```

RECOMMENDED PRACTICES FOR PLAYBOOK MANAGEMENT:

- Some recommended practices for playbook development are listed below:
- Use a concise description of the play's or task's purpose to name plays and tasks. The play name or task name is displayed when the playbook is executed. This also helps document what each play or task is supposed to accomplish, and possibly why it is needed.
- Include comments to add additional inline documentation about tasks.
- Make effective use of vertical white space. In general, organize task attributes vertically to make them easier to read.
- Consistent horizontal indentation is critical. Use spaces, not tabs, to avoid indentation errors. Set up your text editor to insert spaces when you press the Tab key to make this easier.
- Try to keep the playbook as simple as possible. Only use the features that you need.

TROUBLESHOOTING CONNECTIONS

- Many common problems when using Ansible to manage hosts are associated with connections to the host and with configuration problems around the remote user and privilege escalation.
- If you are having problems authenticating to a managed host, make sure that you have **remote_user** set correctly in your configuration file or in your play.
- You should also confirm that you have the correct **SSH keys** set up or are providing the correct password for that user.
- Make sure that **become** is set properly, and that you are using the correct **become_user** (this is root by default). You should confirm that you are entering the correct sudo password and that sudo on the managed host is configured correctly.
- A more subtle problem has to do with inventory settings. For a complex server with multiple network addresses, you may need to use a particular address or DNS name when connecting to that system. You might not want to use that address as the machine's inventory name for better readability. You can set a host inventory variable, **ansible_host**, that will override the inventory name with a different name or IP address and be used by Ansible to connect to that host. This variable could be set in the **host_vars** file or directory for that host, or could be set in the inventory file itself. For example, the following inventory entry configures Ansible to connect to 192.0.2.4 when processing the host

...

```
web4.phx.example.com: web4.phx.example.com ansible_host=192.0.2.4
```

...

Meaning:

- Inventory name: **web4.phx.example.com**
- Ansible connects to: **192.0.2.4**

This is perfectly valid and widely used.

This is a useful way to control how Ansible connects to managed hosts. However, it can also cause problems if the value of **ansible_host** is incorrect.

THE CORRECT LEVEL OF TESTING

- Ansible is designed to ensure that the configuration included in playbooks and performed by its modules is correct.
- It monitors all modules for reported failures, and stops the playbook immediately if any failure is encountered.
- This helps ensure that any task performed before the failure has no errors. Because of this, there is usually no need to check if the result of a task managed by Ansible has been correctly applied on the managed hosts.
- It makes sense to add some health checks either to playbooks, or run those directly as ad hoc commands, when more direct troubleshooting is required. But, you should be careful about adding too much complexity to your tasks and plays in an effort to double check the tests performed by the modules themselves.